Visvesvaraya Technological University, Belagavi – 590018



MAJOR PROJECT PHASE I REPORT
ON

AUTOMATED CARBON EMISSION MONITORING AND ESG REPORTING SYSTEM

Submitted in partial fulfillment for the award of degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

Submitted by

Dion Joshua Lobo	**4SO23CD015**
Pradeep Badiger	**4SO23CD040**
Reyon Joseph	**4SO23CD042**
Shovin Jeson D'Souza	**4SO23CD051**

Under the Guidance of

Dr. Vijetha U.
Associate Professor, Department of ICBS



DEPT. OF INTELLIGENT COMPUTING AND BUSINESS SYSTEMS
ST JOSEPH ENGINEERING COLLEGE
An Autonomous Institution
(Affiliated to VTU Belagavi, Recognized by AICTE, Accredited by NBA)
Vamanjoor, Mangaluru - 575028, Karnataka

2025-26

ST JOSEPH ENGINEERING COLLEGE

An Autonomous Institution

(Affiliated to VTU Belagavi, Recognized by AICTE, Accredited by NBA)

Vamanjoor, Mangaluru - 575028, Karnataka

DEPT. OF INTELLIGENT COMPUTING AND BUSINESS SYSTEMS

CERTIFICATE

Certified that the project work entitled **"AUTOMATED CARBON EMISSION MONITORING AND ESG REPORTING SYSTEM"** carried out by

Dion Joshua Lobo	**4SO23CD015**
Pradeep Badiger	**4SO23CD040**
Reyon Joseph	**4SO23CD042**
Shovin Jeson D'Souza	**4SO23CD051**

the bonafide students of VII semester Computer Science & Engineering(Data Science) in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering (Data Science) of the Visvesvaraya Technological University, Belagavi during the year 2025-2026. It is certified that all suggestions indicated during internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Dr. Vijetha U.	**Dr Harivinod N**	**Dr Rio D'Souza**
Project Guide	HOD-ICBS	Principal

External Viva:

Examiner's Name	Signature with Date
1.	.....
2.	.....

ST JOSEPH ENGINEERING COLLEGE

An Autonomous Institution

(Affiliated to VTU Belagavi, Recognized by AICTE, Accredited by NBA)

Vamanjoor, Mangaluru - 575028, Karnataka

DEPT. OF INTELLIGENT COMPUTING AND BUSINESS SYSTEMS

DECLARATION

We hereby declare that the entire work embodied in this Project Report titled **"AUTOMATED CARBON EMISSION MONITORING AND ESG REPORTING SYSTEM"** has been carried out by us at St Joseph Engineering College, Mangaluru under the supervision of **Dr Harivinod N,** for the award of **Bachelor of Engineering** in **Computer Science & Engineering(Data Science)** . This report has not been submitted to this or any other University for the award of any other degree.

Name	**USN**	**Signature**
Dion Joshua Lobo	4SO23CD015	
Pradeep Badiger	4SO23CD040	
Reyon Joseph	4SO23CD042	
Shovin Jeson D'Souza	4SO23CD051	

Acknowledgement

We sincerely thank our Project guide **Dr. Vijetha U.**, Associate Professor, Computer Science and Engineering for her guidance and valuable suggestions which helped us to complete this project. We also thank our Project coordinator **Ms Farah Anjum**, Dept of ICBS, for their consistant encouragement.

We owe a profound gratitude to **Dr Harivinod N**, Head of the Department, ICBS, whose kind support and guidance helped us to complete this work successfully

We are extremely thankful to our Principal, **Dr Rio D'Souza**, Director, Rev. **Fr Wilfred Prakash D'Souza**, and Assistant Director, **Rev Fr Kenneth Rayner Crasta** for their support and encouragement.

We would like to thank all faculty and staff of the Deprtment of ICBS who have always been with us extending their support, precious suggestions, guidance, and encouragement through the project.

We also extend our gratitude to our friends and family members for their continuous support.

Abstract

Accurate and transparent carbon emission reporting has become a pressing requirement for industries as environmental regulations tighten and ESG accountability grows. Many organizations still depend on manually prepared spreadsheets, self-declared activity data, and disconnected records for their environmental disclosures. This approach slows reporting, makes verification difficult, and limits the ability of decision-makers to identify abnormal emission patterns in time.

This project presents CarbonTrace, a full-stack web application for automated carbon emission monitoring and ESG reporting. The system uses a FastAPI backend, PostgreSQL persistence, and a React dashboard to manage facilities, users, activity records, emission factors, calculated emissions, anomalies, recommendations, and scenario data. It supports authenticated CSV upload processing, role-based access control, token refresh handling, dashboard data APIs, anomaly-analysis boundaries, and downloadable PDF ESG report generation.

The implementation is designed as a modular and deployable platform. Docker Compose coordinates PostgreSQL, the backend, frontend, and Nginx reverse proxy, while the frontend provides protected pages for live monitoring, trends, anomaly review, cross-verification, and report download. By replacing fragmented manual reporting with a centralized and auditable application, CarbonTrace improves data traceability, supports ESG disclosure workflows, and provides a foundation for future expansion into direct sensor ingestion and broader sustainability metrics.

Table of Contents

Acknowledgement **i**

Table of Contents **v**

List of Figures **vi**

List of Tables **vii**

1 Introduction **1**

1.1 Background 1
1.2 Problem Statement 1
1.3 Scope 2
1.4 Objectives 2

2 Literature Survey **3**

2.1 Introduction 3
2.2 Leveraging IoT and Deep Learning for Real-Time Carbon Footprint Monitoring and Optimization in Smart Cities and Industrial Zones . . . 3
2.3 Green IoT Event Detection for Carbon-Emission Monitoring in Sensor Networks 4
2.4 Estimating CO_2 Emissions from IoT Traffic Flow Sensors and Reconstruction 4
2.5 IoT-Based Real-Time Environment Monitoring and Safety for Factory Workplace 5
2.6 The Power of IoT in ESG Data Collection and Analysis 5
2.7 Building an IoT Data Hub with Elasticsearch, Logstash and Kibana . . 6
2.8 A New Approach Based on ELK Stack for the Analysis and Visualisation of Geo-referenced Sensor Data 6
2.9 Real-Time Air Quality Monitoring Using IoT Sensors 7
2.10 Using the Integral Digital Twin for Product Carbon Footprint Calculation 7
2.11 Internet of Things' Sustainability Effects: Quantile and Temporal Insights 8
2.12 Comparative Analysis of Reviewed Literature 8
2.13 Summary 10
2.14 Proposed System 10

3 Software Requirements Specification **12**
3.1 Functional Requirements . . . 12
3.2 Non-Functional Requirements . . . 13
3.3 User Interface Design . . . 14
3.3.1 Core Components . . . 14
3.3.2 Interface Characteristics . . . 15
3.4 Hardware and Software Requirements . . . 15
3.4.1 Software Requirements . . . 15
3.4.2 Hardware Requirements . . . 16
3.5 Performance Requirements . . . 16
3.6 Design Constraints . . . 17
3.7 Any Other Requirements . . . 17
3.7.1 Data Quality and Governance . . . 17
3.7.2 Future Enhancements . . . 17

4 System Design **18**
4.1 Abstract Design . . . 18
4.1.1 Architectural Diagram . . . 18
4.2 Control Flow Design . . . 19
4.2.1 Algorithms for Logic Implementation . . . 19
4.3 Access Layer Design . . . 19
4.3.1 ER Model . . . 19
4.3.2 Database Schema . . . 20
4.4 Presentation Layer Design . . . 20
4.4.1 User Interface Overview . . . 20

5 Implementation **21**
5.1 System-Level Implementation . . . 21
5.2 Backend Implementation . . . 22
5.2.1 Application Startup and Routing . . . 22
5.2.2 Configuration Management . . . 22
5.2.3 Database Session and Initialization . . . 22
5.2.4 Data Model Implementation . . . 22
5.2.5 Authentication and Authorization . . . 23
5.2.6 CSV Upload and Data Ingestion . . . 23
5.2.7 OCR Draft Extraction . . . 24
5.2.8 Emission Calculation Layer . . . 24
5.2.9 Emission Queries, Anomaly Detection, and Recommendations . . . 24
5.2.10 Scenario Simulation . . . 25
5.2.11 PDF ESG Report Output . . . 25
5.3 Frontend Implementation . . . 25
5.3.1 Application Routes and Protected Layout . . . 25

5.3.2 Frontend Authentication Flow 26
5.3.3 Dashboard Data Fetching 26
5.3.4 Page-Level Features 26
5.4 Deployment Implementation 27
5.4.1 Docker Compose Services 27
5.4.2 Backend and Frontend Images 27
5.4.3 Reverse Proxy and Environment Variables 27
5.5 Implementation Summary 27

6 System Testing **28**
6.1 Testing Strategy 28
6.2 Test Cases 28
6.3 Development Workflow 28

7 Results and Discussion **30**
7.1 Application-Level Results 30
7.2 Backend Results 30
7.3 Frontend Results 31
7.4 Deployment Results 31
7.5 Discussion 31

8 Conclusion and Future Work **32**

References **33**

A Screenshots **34**

B User Manual **35**
B.1 Starting the Application 35
B.2 Signing In 35
B.3 Using the Dashboard 35
B.4 Uploading Data 35
B.5 Signing Out 36

List of Figures

4.1 Sequence Diagram for Data Processing 18
4.2 Simplified ER Model 19

List of Tables

2.1 Traditional Approach vs. Proposed System 9
2.3 Comparative Analysis of Reviewed Literature 9

3.1 Functional Requirements of the Proposed System 12
3.3 Non-Functional Requirements of the Proposed System 13
3.5 Software Requirements 15
3.7 Hardware Requirements 16

4.1 Database Schema Summary 20

5.1 Major implementation components in CarbonTrace 21
5.2 Implemented backend data models 23

6.1 System Test Cases 29
6.2 Implementation Workflow 29

Chapter 1

Introduction

1.1 Background

Industrial activity contributes significantly to global carbon emissions, and the demand for transparent environmental reporting continues to increase as organizations adopt net-zero targets and ESG disclosure practices. Accurate measurement of electricity use, fuel consumption, and facility-level activity data is now essential for both internal sustainability planning and external compliance reporting.

In India, environmental reporting is influenced by regulatory bodies such as the Central Pollution Control Board (CPCB) and State Pollution Control Boards (SPCBs). Many organizations still prepare emission summaries through manual collection, spreadsheet consolidation, and periodic self-reporting. These methods are vulnerable to missing values, delayed submissions, inconsistent units, and weak audit trails.

CarbonTrace addresses this gap by providing a centralized web application for carbon emission monitoring and ESG reporting. The system combines a FastAPI backend, PostgreSQL database, React dashboard, and containerized deployment. It is designed to ingest activity data, validate uploaded records, calculate carbon dioxide equivalent values using emission factors, display trends and anomalies, and generate structured PDF reports.

1.2 Problem Statement

Industries require a reliable way to consolidate facility activity data, emission calculations, anomaly checks, and ESG reporting into one auditable workflow. Existing manual processes make it difficult to verify the source of reported values, compare records across data sources, detect unusual emission patterns, and produce consistent reports for review.

The project therefore focuses on building a secure and modular platform that can store facility data, manage role-based users, process uploaded activity records, calculate emissions, expose dashboard APIs, and support ESG report generation. The platform must also be deployable as a complete application so that backend, frontend, database,

and reverse-proxy services can work together reliably.

1.3 Scope

This project covers the design and development of CarbonTrace as a full-stack carbon monitoring and ESG reporting application. The implemented scope includes backend APIs for authentication, facilities, uploads, emissions, activity records, calculations, anomalies, recommendations, scenarios, and reports. PostgreSQL is used as the main transactional store for users, roles, sessions, facilities, uploads, activity records, emission factors, calculated emissions, recommendations, and scenario records.

The frontend scope includes a protected React dashboard with pages for live monitoring, trend analysis, anomaly review, cross-verification, ESG report download, and login. The deployment scope includes Docker Compose services for PostgreSQL, backend, frontend, and Nginx, with environment-based configuration for database access, JWT settings, CORS origins, and API routing.

1.4 Objectives

- To develop a secure full-stack platform for carbon emission monitoring and ESG reporting.
- To model facilities, users, roles, activity records, emission factors, calculated emissions, recommendations, and scenarios in PostgreSQL.
- To implement authenticated backend APIs using FastAPI, JWT access tokens, HTTP-only refresh cookies, and role-based route protection.
- To support CSV upload validation and provide a clear path for manual entry and reviewed OCR-derived activity records.
- To calculate carbon dioxide equivalent values from activity quantities and emission factors while preserving calculation traceability.
- To provide dashboard views for live summaries, trends, anomalies, cross-verification, and ESG report download.
- To package the application with Docker Compose and Nginx so it can be run as an integrated deployment.

Chapter 2

Literature Survey

2.1 Introduction

The growing urgency of climate change and the increasing adoption of Environmental, Social, and Governance (ESG) frameworks by corporations have driven significant research interest in automated emission monitoring systems. Traditional methods of carbon accounting rely on manual data collection, periodic audits, and spreadsheet-based reporting, which are error-prone, slow, and difficult to scale. With the proliferation of Internet of Things (IoT) sensors, edge computing, and real-time data analytics platforms, it has become feasible to design automated, continuous, and transparent carbon emission monitoring systems.

This chapter critically reviews ten peer-reviewed works and research articles that form the foundational basis for the proposed system. The reviewed works span four key themes: IoT-based environmental and emission monitoring, ESG data collection and reporting, ELK Stack (Elasticsearch, Logstash, and Kibana) data pipeline architectures, and digital twin and sustainability-driven data infrastructure. Each work is analyzed with respect to its problem statement, methodology, results, and limitations. A comparative analysis is presented at the end of the chapter, followed by a discussion of the gap addressed by the proposed system.

2.2 Leveraging IoT and Deep Learning for Real-Time Carbon Footprint Monitoring and Optimization in Smart Cities and Industrial Zones

[1] Ojadi et al. address the challenge of continuous and accurate carbon footprint monitoring in smart cities and industrial environments. The authors observe that conventional monitoring approaches are reactive and provide only aggregated, retrospective emissions data, making proactive climate action difficult. The motivation for this work stems from the rapid expansion of smart city infrastructure and the need for real-time decision support for emission reduction.

The proposed system integrates IoT sensors deployed across target zones with deep

learning models for pattern recognition and anomaly detection. Gas sensors measure CO_2, CH_4, and NO_x concentrations at edge nodes, while a centralized deep neural network processes incoming streams to predict emission trends and flag unusual spikes. The authors validate the system in a simulated smart city environment and demonstrate that deep learning significantly improves anomaly detection accuracy over rule-based baselines.

The work establishes a strong precedent for combining IoT sensing with intelligent analytics in emission monitoring contexts. However, the system does not include a structured ESG reporting layer or a standardized output format suitable for regulatory disclosure. The proposed system in this project extends this idea by combining structured activity data, backend validation, emission-factor-based calculation, and PDF ESG report generation.

2.3 Green IoT Event Detection for Carbon-Emission Monitoring in Sensor Networks

[2] Fay, Corcoran, and Diamond propose an event-driven sensing paradigm specifically optimized for carbon-emission monitoring in low-power IoT networks. The authors identify that continuous polling of gas sensors leads to high energy consumption, which is counterproductive for green computing objectives. They propose an event-detection framework wherein sensors transmit data only when statistically significant emission events occur.

The methodology employs adaptive thresholding and statistical change-point detection algorithms embedded at the sensor firmware level. The system was evaluated on a testbed of wireless gas sensors measuring CO_2 and particulate matter, demonstrating substantial reductions in data transmission volume without compromising detection accuracy. The authors show that energy consumption decreases by up to 60% compared to periodic polling approaches.

This work highlights an important design consideration for large-scale IoT deployments: the trade-off between data granularity and energy efficiency. For the proposed system, the event-driven model informs the ingestion strategy by encouraging validation and prioritization before activity records are stored or used in emission calculations.

2.4 Estimating CO_2 Emissions from IoT Traffic Flow Sensors and Reconstruction

[3] Badii et al. present a methodology for estimating vehicular CO_2 emissions using data from IoT-based traffic flow sensors deployed in urban environments. The work addresses the problem of deriving indirect emissions, where direct measurement is impractical, from observable proxies such as vehicle count, speed, and road classification.

The authors develop an emission reconstruction algorithm that correlates traffic flow

metrics obtained from roadside IoT sensors with emission factors defined in standard datasets such as COPERT. The approach is validated against direct measurement stations in an Italian city and achieves a high degree of accuracy in CO_2 estimation at the road-segment level. Kibana dashboards are used as the visualization layer for aggregated emission maps.

This study is particularly relevant because it demonstrates the viability of using indirect IoT measurement for emission quantification, which is a core assumption of the proposed system. The system similarly supports activity data such as electricity use and fuel consumption rather than requiring direct gas measurement in all scenarios. The visualization approach also supports the need for a clear dashboard layer for operational and compliance users.

2.5 IoT-Based Real-Time Environment Monitoring and Safety for Factory Workplace

[4] Suryawanshi et al. focus on developing a real-time industrial environment monitoring system using IoT, targeting factory workplaces where hazardous gas concentrations, temperature, and particulate matter pose occupational health risks. The authors emphasize that conventional spot-check monitoring is insufficient for dynamic industrial environments where emission levels can spike rapidly.

The system architecture consists of a distributed sensor network connected via MQTT to a cloud-based data broker. A web dashboard provides live visualization and threshold-based alerting. The paper evaluates the system's response latency and alert accuracy in a manufacturing plant context, demonstrating sub-second alert delivery for dangerous gas level exceedances.

This work reinforces the choice of MQTT as a lightweight, reliable messaging protocol for IoT data transmission in the proposed system. The factory monitoring scenario is also directly applicable to the industrial carbon emission monitoring use case, where the same sensor types such as CO_2, CO, temperature, and humidity are deployed to track operational emissions and workplace compliance.

2.6 The Power of IoT in ESG Data Collection and Analysis

[5] Salehi examines the strategic role of IoT in modernizing ESG data collection frameworks, arguing that manual data gathering processes introduce significant reliability and consistency issues in corporate sustainability reporting. The paper surveys existing IoT deployments across energy, water, waste, and emissions monitoring use cases and evaluates their contribution to ESG data quality.

The author finds that IoT-enabled continuous monitoring substantially improves data completeness and timeliness compared to quarterly or annual manual audits. Specific applications discussed include smart energy meters for Scope 1 and Scope 2 emis-

sions tracking, connected waste sensors for circular economy metrics, and air quality monitors for environmental compliance. The paper also identifies the challenge of data governance, standardization, and integration with existing ESG reporting frameworks such as GRI, SASB, and TCFD.

This work provides the theoretical and practical justification for a data-driven monitoring architecture. The integration challenges identified, particularly around data standardization and mapping to ESG reporting frameworks, directly motivate the proposed system's use of backend validation, normalized PostgreSQL models, and structured report-generation services.

2.7 Building an IoT Data Hub with Elasticsearch, Logstash and Kibana

[6] Bajer presents a foundational implementation of the ELK Stack as a data hub for processing and visualizing IoT data, demonstrating that tools originally designed for log management can be effectively repurposed for storing, searching, and visualizing heterogeneous sensor data. The work is motivated by the rapid growth of distributed IoT deployments and the need for a scalable, open-source infrastructure to consolidate and make sense of the large volumes of data they produce.

The implementation uses data generated by building subsystems at ABB Corporate Research in Krakow, including HVAC, lighting, and energy sensors, integrated into a single Elasticsearch-based solution via Logstash. The paper describes the full three-stage Logstash pipeline (inputs $\rightarrow$ filters $\rightarrow$ outputs), Elasticsearch's shard-based distributed indexing, and a custom Kibana plugin that subscribes directly to an MQTT broker via WebSocket to display live sensor readings on an SVG floor plan. Horizontal and vertical scaling strategies for growing IoT deployments are also discussed, along with a comparison of alternative stacks including Splunk, Graylog, InfluxDB, and Grafana.

This paper is an important architectural precedent for continuous environmental data platforms. The MQTT-to-ELK data flow described by Bajer separates ingestion, transformation, storage, and visualization responsibilities. The proposed carbon emission monitoring system adopts the same separation of concerns, implemented through FastAPI services, PostgreSQL persistence, and a React dashboard.

2.8 A New Approach Based on ELK Stack for the Analysis and Visualisation of Geo-referenced Sensor Data

[7] Ngo, Sarramia, Kang, and Pinet examine the use of the ELK Stack as a spatial data warehouse for analysing and visualising geo-referenced environmental sensor data. The work is motivated by the French CEBA environmental data lake project, which collects heterogeneous environmental measurements including air temperature, soil qual-

ity, and water parameters from distributed sensor networks across the Auvergne region, and requires a unified, queryable infrastructure to support multi-disciplinary scientific analysis.

The authors propose a multidimensional data model implemented in Elasticsearch, extending the standard ELK architecture with a streaming ETL component built on top of Logstash that performs real-time aggregation and integration of data from heterogeneous sensor sources. The paper evaluates Elasticsearch's querying performance for time-series and spatial data at scale, benchmarking it against competing solutions and demonstrating that Elasticsearch handles analytical queries on large sensor datasets with acceptable latency. Kibana is used to build per-sensor and per-location dashboards with time-range filtering.

This study is highly relevant to the proposed system for two reasons. First, it validates the need for a scalable backend capable of handling continuous, multi-source environmental data. Second, the streaming ETL architecture demonstrates the importance of normalization between raw inputs and stored reporting records, which directly informs the proposed system's validation and activity-record design.

2.9 Real-Time Air Quality Monitoring Using IoT Sensors

[8] Narute, Navagire, and Konge propose an IoT-based urban air quality monitoring system designed to address the high cost and limited spatial coverage of traditional stationary air quality stations. The system employs a hybrid sensor network consisting of both fixed and mobile sensor nodes to achieve wider urban coverage at reduced deployment cost.

The architecture uses low-cost gas sensors for measuring pollutants including $PM_{2.5}$, CO, and NO_2, connected via Wi-Fi and LoRa to a central data aggregation server. The collected data is processed and visualized on a web-based dashboard, providing city planners and policymakers with granular, real-time air quality maps. The authors validate the system's accuracy against reference-grade monitoring stations and report acceptable correlation coefficients for all measured pollutants.

This work directly informs the sensor selection and network topology considerations of the proposed system. The hybrid fixed-mobile sensor approach is applicable in large industrial or campus environments where full wired sensor coverage is impractical. The dashboard design concepts, particularly spatial summaries and temporal trend charts, inform the React visualization layer of the proposed system.

2.10 Using the Integral Digital Twin for Product Carbon Footprint Calculation

[9] Winter et al. investigate the application of digital twin technology for calculating and managing Product Carbon Footprints (PCF) across the product lifecycle. The

authors argue that static lifecycle assessment (LCA) methods fail to capture dynamic changes in manufacturing processes, material sourcing, and energy mix, and that digital twins offer a continuously updated, data-driven alternative.

The proposed digital twin framework integrates sensor data from production lines, supplier data from procurement systems, and energy consumption data from smart meters into a unified PCF calculation engine. The system updates carbon footprint estimates in near real-time as process conditions change, enabling product designers and operations managers to make data-driven decisions for emission reduction.

This study expands the conceptual scope of the proposed system by demonstrating that IoT-driven carbon monitoring need not be limited to facility-level Scope 1 emissions. The digital twin approach, where physical asset data is continuously mirrored in a computational model, parallels the proposed system's separation between official activity records, calculated emissions, and scenario simulations. The PCF reporting outputs described in this work also inform the ESG report generation module of the proposed system.

2.11 Internet of Things' Sustainability Effects: Quantile and Temporal Insights

[10] Li and Lao present a rigorous econometric analysis of the relationship between IoT adoption and carbon emission reduction, employing quantile regression and panel data analysis across a multi-country dataset. The authors seek to quantify whether and to what extent IoT deployments contribute to measurable sustainability improvements, controlling for economic development level, energy mix, and industrial structure.

The study finds that the emission-reduction effect of IoT is heterogeneous and nonlinear: the impact is stronger in higher-emission economies and exhibits temporal dynamics where benefits accumulate over time as IoT infrastructure matures and data utilization deepens. The authors conclude that IoT contributes to sustainability through three primary channels, namely energy efficiency optimization, process automation, and enabling data-driven decision-making.

This empirical study provides macroeconomic validation for the assumptions underlying the proposed system. The finding that data utilization deepens IoT's sustainability impact over time directly supports the need for a centralized data platform with historical retention, trend analysis, and reporting over accumulated activity and emission data. The temporal insights also motivate the proposed system's support for longitudinal ESG reporting.

2.12 Comparative Analysis of Reviewed Literature

The reviewed works collectively establish a strong technical and conceptual foundation for the proposed system. Table 2.1 first summarizes how the proposed system compares

against traditional approaches at a high level, and Table 2.3 then provides a detailed cross-paper comparison across the key dimensions addressed by each reviewed work.

Table 2.1: Traditional Approach vs. Proposed System

Evaluation Metric	**Traditional Approach**	**Proposed System**
Data Collection	Manual logging and periodic sampling	Structured activity data through uploads, manual entry, and future meter integrations
Processing Latency	High (days to weeks)	Low through backend validation, calculation services, and dashboard APIs
Scalability	Limited by human resources	Scalable through modular services and PostgreSQL-backed storage
ESG Compliance	Prone to errors, hard to audit	Immutable, structured, audit-ready pipelines

Table 2.3: Comparative Analysis of Reviewed Literature

Paper	**IoT Sensing**	**Real-Time Processing**	**ESG / Reporting**	**Analytics / AI**	**Visualization**
Ojadi et al. [1]	✓	✓	×	Deep Learning	×
Fay et al. [2]	✓	✓	×	Statistical	×
Badii et al. [3]	✓	Partial	×	Regression	Kibana
Suryawanshi et al. [4]	✓	✓	×	Rule-based	Dashboard
Salehi [5]	✓	Partial	✓	Survey	×
Bajer [6]	✓	✓	×	ELK Pipeline	Kibana
Ngo et al. [7]	✓	✓	×	ELK + ETL	Kibana
Narute et al. [8]	✓	✓	×	Statistical	Dashboard
Winter et al. [9]	✓	✓	✓	Digital Twin	×
Li & Lao [10]	×	×	✓	Econometric	×
Proposed System	✓	✓	✓	**Rules + ML**	**React**

From the comparative analysis, a clear research gap emerges. While several works

address IoT-based emission sensing and others address ESG reporting frameworks or data-pipeline architectures in isolation, few systems combine facility data management, validated activity ingestion, emission-factor-based calculation, anomaly analysis, dashboard visualization, and automated structured ESG report generation within a unified architecture. The proposed system addresses this gap by combining backend APIs, PostgreSQL persistence, a React dashboard, and a report generation module that maps aggregated emission metrics to ESG disclosure needs.

2.13 Summary

This chapter reviewed ten works spanning IoT-based emission monitoring, ESG reporting, data infrastructure, and sustainability-focused analytics. Prior systems establish the value of automated data capture, scalable processing, visualization, and analytics for environmental monitoring. The review shows that individual components of the proposed system have been explored in prior work, but an integrated application combining facility management, validated activity ingestion, emission calculation, anomaly review, dashboard visualization, and automated ESG report generation remains a clear need. The proposed Automated Carbon Emission Monitoring and ESG Reporting System is positioned to fill this gap, offering transparent and auditable emission reporting for industrial and commercial organizations.

2.14 Proposed System

Based on the gaps identified in the reviewed literature, this project proposes an automated carbon emission monitoring and ESG reporting system that combines structured activity-data ingestion, relational storage, backend calculation services, dashboard visualization, and PDF reporting. The core idea is to replace fragmented manual reporting processes with a centralized workflow that is transparent and verifiable.

The proposed system is organized across four layers, each handling a distinct stage of the data flow.

The first is the **data collection layer**, where activity data is gathered from CSV uploads, manual records, reviewed OCR outputs, and future meter or IoT integrations. This layer preserves source metadata so that reported values can be traced back to their origin.

The second is the **data processing layer**, implemented through FastAPI service modules. This layer validates uploaded data, normalizes activity records, selects matching emission factors, and calculates carbon dioxide equivalent values.

The third is the **storage and verification layer**, handled through PostgreSQL-backed models and query services. Activity records, calculated emissions, uploads, facilities, recommendations, and scenarios are stored in relational tables, while anomaly and cross-verification logic compares records across sources and time periods.

The fourth is the **ESG reporting and visualization layer**, delivered through a React

dashboard and PDF report generator. The dashboard is designed for operational users who need monitoring views and for compliance users who need structured, audit-ready summaries for review.

The system is designed to be modular and scalable, and can be extended to cover additional ESG parameters or deployed across multiple facilities without significant rework. Overall, the proposed system addresses the key shortcomings found in the reviewed literature by offering real-time visibility, automated validation, and a centralized platform that makes environmental reporting both more reliable and more actionable.

Chapter 3

Software Requirements Specification

3.1 Functional Requirements

This section lists the functional capabilities required for CarbonTrace. Each requirement is written as a verifiable feature that can be checked through backend APIs, frontend screens, or deployment tests.

Table 3.1: Functional Requirements of the Proposed System

ID	Feature	Requirement
FR1	User Authentication	The system shall authenticate users through email and password credentials and issue a short-lived JWT access token with a secure refresh-token cookie.
FR2	Role-Based Access Control	The system shall restrict protected operations according to assigned roles such as administrator, facility manager, and auditor.
FR3	Facility Management	The system shall store facility details including name, location, region code, coordinates, and facility type.
FR4	Multi-Source Data Ingestion	The system shall support activity data from manual entry, CSV upload, and reviewed OCR extraction workflows.
FR5	CSV Upload Validation	The system shall validate uploaded CSV files for required fields, supported formats, timestamp parsing, and numeric activity values before accepting the records for processing.

ID	Feature	Requirement
FR6	Emission Factor Management	The system shall maintain emission factors by activity type, region, unit, and validity period so that calculations remain traceable and reproducible.
FR7	Emission Calculation	The system shall calculate carbon dioxide equivalent values by applying the matching emission factor to a validated activity record and storing the result with the factor reference.
FR8	Dashboard Monitoring	The system shall provide dashboard APIs and frontend views for latest readings, emission summaries, time-series trends, anomaly review, and cross-verification.
FR9	ESG Report Generation	The system shall generate a downloadable PDF ESG report containing emission summaries, anomaly information, and report metadata.
FR10	Scenario Simulation	The system shall allow hypothetical scenario records to be stored separately from official activity and calculated-emission records.

3.2 Non-Functional Requirements

The following requirements define the measurable quality attributes expected from the system.

Table 3.3: Non-Functional Requirements of the Proposed System

ID	Quality Attribute	Requirement
NF1	Usability	A compliance user shall be able to navigate to the dashboard, review emission trends, inspect anomalies, and download an ESG report after a short onboarding session.
NF2	Performance	Dashboard API endpoints for summary and trend views shall respond within an acceptable time for normal reporting periods and refresh periodically without manual reload.
NF3	Reliability	Backend services shall close database sessions after each request and handle failed validation paths without corrupting persisted records.

ID	Quality Attribute	Requirement
NF4	Availability	The containerized services shall run together through Docker Compose, with PostgreSQL health checks used before backend startup.
NF5	Scalability	The architecture shall allow additional facilities, activity sources, emission factors, dashboard queries, and ESG metrics to be added without redesigning the entire system.
NF6	Security	Passwords shall be hashed before storage, refresh tokens shall be stored only as hashes, and production startup shall reject the default development JWT secret.
NF7	Maintainability	Backend routers, service modules, frontend pages, and deployment configuration shall remain separated so that each layer can be updated independently.
NF8	Compatibility	The frontend dashboard shall render on common modern browsers and remain usable on standard laptop displays.
NF9	Data Integrity	Official activity records and calculated emissions shall remain separate from draft OCR values and hypothetical scenario data.

3.3 User Interface Design

3.3.1 Core Components

The user interface is built as a protected React dashboard. It is organized around the main workflows required by facility managers, auditors, and administrators.

Live Monitoring Dashboard: The primary screen gives users a consolidated view of current emission summaries and latest readings. It is designed for quick inspection of facility-level carbon activity.

Trends Page: This screen presents time-series emission data and allows users to review changes over selected metrics and intervals.

Anomaly Page: This screen lists records flagged by anomaly-analysis logic and provides supporting details such as the metric, facility, score, and expected value when available.

Cross-Verification Page: This screen compares readings across different sources so users can identify mismatches between manual, uploaded, OCR-derived, or calculated records.

ESG Report Page: This screen allows users to request and download the generated PDF ESG report.
User Session Controls: The layout includes authenticated navigation, user identity display, and logout handling.

3.3.2 Interface Characteristics

- The dashboard shall use clear summary cards, tables, charts, and status indicators to reduce interpretation effort.
- Protected routes shall redirect unauthenticated users to the login page.
- Dashboard data shall refresh periodically for live-monitoring behavior.
- Navigation shall remain consistent across monitoring, trends, anomalies, cross-verification, and reporting pages.
- The interface shall distinguish official reporting data from draft or scenario data where both are shown.

3.4 Hardware and Software Requirements

3.4.1 Software Requirements

Table 3.5: Software Requirements

Component	**Specification**
Backend Framework	FastAPI for REST API development, request validation, routing, and dependency injection.
Database	PostgreSQL for transactional storage of users, sessions, facilities, activity records, emission factors, calculated emissions, recommendations, and scenarios.
ORM and Validation	SQLAlchemy for database models and Pydantic for settings and request or response schemas.
Frontend	React with Vite, React Router, TanStack Query, Axios, Tailwind CSS, Recharts, and Lucide icons.
Authentication	JWT access tokens, HTTP-only refresh-token cookies, bcrypt password hashing, and role-based backend dependencies.
Reporting	ReportLab for PDF ESG report generation.
Containerization	Docker Compose for PostgreSQL, backend, frontend, and Nginx service orchestration.

Component	Specification
Version Control and CI	Git and GitHub Actions for source control, lint checks, backend import validation, dependency installation, and frontend build validation.

3.4.2 Hardware Requirements

Table 3.7: Hardware Requirements

Component	Specification
Development System	Laptop or desktop with a multi-core processor and at least 16 GB RAM for running backend, frontend, database, and containers during development.
Server Infrastructure	A Linux-based server or virtual machine capable of running Docker Compose services for the application stack.
Storage	Sufficient persistent disk space for PostgreSQL data, uploaded file metadata, application logs, and generated reports.
Network	Stable network connectivity between browser clients and the hosted application endpoint.
Optional Data Devices	IoT meters, smart energy meters, or facility data-export systems may be integrated later as additional activity data sources.

3.5 Performance Requirements

- Authentication and user profile endpoints shall respond quickly enough to support normal dashboard session startup.
- Dashboard summary, latest, and time-series endpoints shall support periodic frontend refresh without noticeably blocking the interface.
- CSV validation shall return clear feedback for invalid files and process normal facility data files within a practical upload workflow.
- PDF report generation for a standard reporting period shall complete within an acceptable download time for compliance users.
- The frontend production build shall produce static assets that can be served efficiently by Nginx.

3.6 Design Constraints

- PostgreSQL is used as the primary data store to keep reporting data, authentication records, and calculation outputs in one relational system.
- Docker Compose is used for deployment because it provides repeatable local and server execution of the database, backend, frontend, and reverse proxy.
- Public sign-up is excluded so that user creation remains controlled by administrators.
- OCR-derived values are treated as drafts until reviewed by a user, preventing unverified extraction errors from entering official reporting tables.
- Scenario values are stored separately from official activity and calculated-emission tables to protect audit data.

3.7 Any Other Requirements

3.7.1 Data Quality and Governance

- Every calculated emission shall keep references to the activity record and emission factor used during calculation.
- Refresh-token sessions shall be revocable so that compromised or expired sessions can no longer be used.
- Uploaded data shall preserve enough metadata to trace who uploaded the file, when it was uploaded, and which facility it relates to.

3.7.2 Future Enhancements

- Direct integration with IoT meters and utility data APIs can be added after the core upload and calculation pipeline is complete.
- Additional ESG categories such as water consumption, waste generation, and energy intensity can be added using the same facility and reporting structure.
- Predictive analytics can be trained on historical activity and calculated-emission data to forecast future threshold breaches.
- Mobile access for field engineers can be introduced for facility checks, meter readings, and maintenance notes.

Chapter 4

System Design

The system design converts the requirements in Chapter 3 into an implementable blueprint for the *Automated Carbon Emission Monitoring and ESG Reporting System*. To keep this chapter focused and relevant, only the most important design artifacts are retained.

4.1 Abstract Design

4.1.1 Architectural Diagram

The architecture follows a layered pattern to keep data acquisition, validation, analytics, and reporting loosely coupled. This separation makes the system easier to scale and maintain.

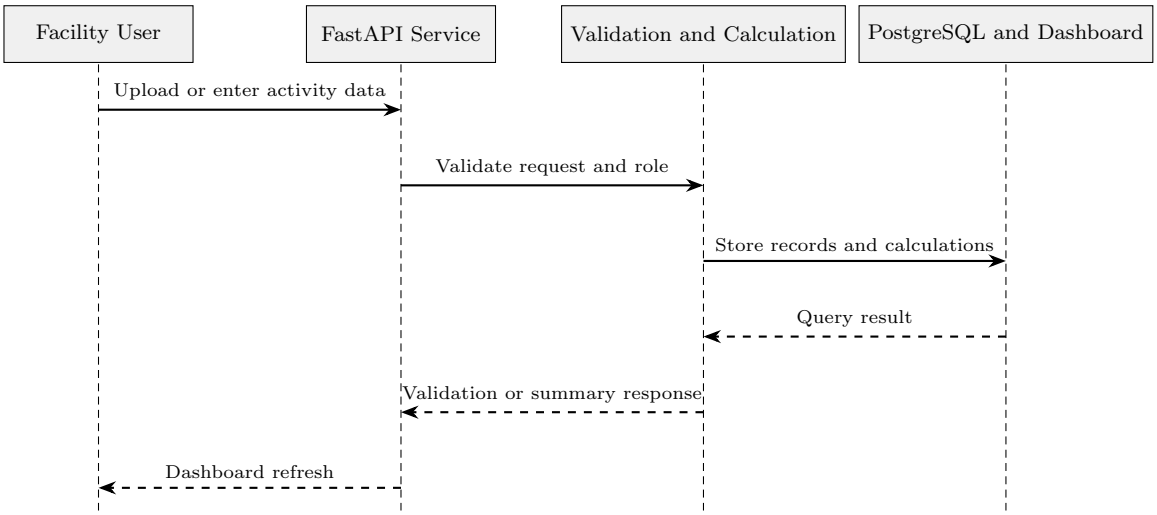


Figure 4.1: Sequence Diagram for Data Processing

4.2 Control Flow Design

4.2.1 Algorithms for Logic Implementation

Algorithm 1: Validation and Normalization

1. Receive record `r` from manual entry, CSV upload, or reviewed OCR output.
2. Validate mandatory fields and data type rules.
3. If invalid, return a validation error with clear reason codes.
4. Normalize valid records to the internal activity-record schema.
5. Store upload metadata and forward validated activity data for calculation.

Algorithm 2: Emission Calculation and Anomaly Detection

1. Select the emission factor matching activity type, region, unit, and validity period.
2. Multiply the activity quantity by the factor value to compute `co2e_kg`.
3. Store the calculated emission with references to the source activity and factor.
4. Compare recent readings for the same facility and metric to identify abnormal values.
5. Return anomaly details and recommendation inputs to the dashboard layer.

4.3 Access Layer Design

4.3.1 ER Model

The ER diagram focuses on the five main entities and only the most important relationships, so it is easier to explain in a report or presentation.

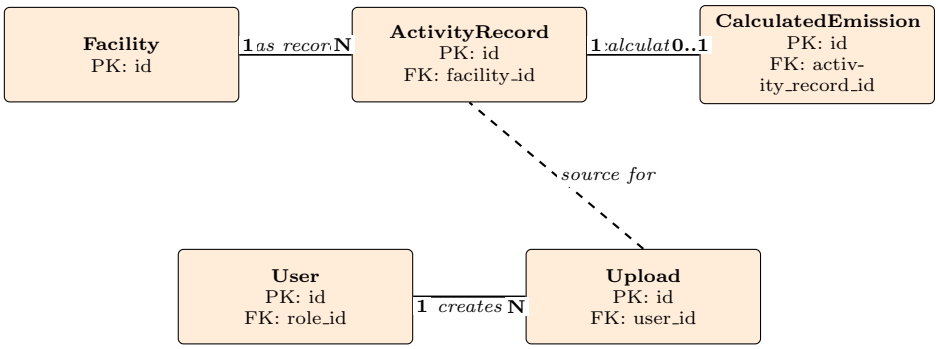


Figure 4.2: Simplified ER Model

This ER diagram can be read in a simple way:

- A **Facility** has many **ActivityRecord** entries.
- An **ActivityRecord** can produce one **CalculatedEmission** result.

- A **User** creates many **Upload** records.
- An **Upload** can act as the source for one or more activity records.

4.3.2 Database Schema

The access layer schema is summarized in Table 4.1.

Table 4.1: Database Schema Summary

Table	Primary Fields	Purpose
`facilities`	id (PK), name, location, region_code, facility_type	Stores registered facility master data.
`activity_records`	id (PK), facility_id (FK), activity_type, quantity, unit, source	Stores measured activity inputs before calculation.
`emission_factors`	id (PK), activity_type, region_code, unit, factor_value	Stores calculation factors and validity periods.
`calculated_emissions`	id (PK), activity_record_id (FK), factor_id (FK), scope, co2e_kg	Stores calculated carbon dioxide equivalent values.
`users`, `roles`, `sessions`	id (PK), role_id (FK), email, token_hash	Store user identity, authorization roles, and refresh sessions.
`uploads`, `recommendations`, `scenarios`	id (PK), facility_id (FK), user_id (FK), period data	Store upload metadata, recommendation outputs, and what-if simulations.

4.4 Presentation Layer Design

4.4.1 User Interface Overview

The presentation layer provides role-based dashboards and guided actions:

- **Facility Manager UI:** upload status, validation errors, live summaries, and anomaly review.
- **Auditor UI:** filtered dashboards, cross-verification views, and report export.
- **Administrator UI:** user-role management and facility configuration.

This interface strategy keeps frequent operational tasks quick while preserving auditability for compliance workflows.

Chapter 5

Implementation

This chapter explains how I built CarbonTrace, a web application for tracking carbon emissions and producing ESG reports. I implemented it as a full-stack system with a FastAPI backend, PostgreSQL for persistence, a React dashboard, and a Docker-based deployment setup using Docker Compose and Nginx.

5.1 System-Level Implementation

At the project level, I separated the application into backend, frontend, database, and reverse-proxy layers. This structure made the project easier to develop and test because each part has a clear responsibility, while Docker Compose still allows the whole system to run together as one application.

Table 5.1: Major implementation components in CarbonTrace

Component	**Implementation details**
Backend API	Built in `backend/` with FastAPI, SQLAlchemy 2.0, Pydantic v2, JWT authentication, role-based route protection, CSV upload handling, anomaly-detection logic, and PDF report output.
Database layer	Uses PostgreSQL to store users, roles, refresh sessions, facilities, uploads, activity records, emission factors, calculated emissions, recommendations, and scenarios.
Frontend dashboard	Built in `frontend/` with React 18, Vite, Tailwind CSS, React Router, TanStack Query, Axios, Recharts, and Lucide icons.
Deployment	Uses Docker Compose to run PostgreSQL, the backend, the frontend, and Nginx. Nginx gives the browser one public entry point and forwards API requests to the backend.
CI workflow	Uses GitHub Actions to synchronize backend dependencies, run Ruff linting, check backend imports, install frontend dependencies, and build the production frontend.

Most application data is stored in PostgreSQL. The backend exposes this data through FastAPI routes, and the React frontend accesses those routes through a shared Axios client and TanStack Query hooks.

5.2 Backend Implementation

5.2.1 Application Startup and Routing

The backend entry point is `backend/main.py`. The `create_app()` function creates the FastAPI app, loads configuration values, checks production safety settings, enables CORS, initializes the database during lifespan startup, and mounts the API routers under the `/api` prefix.

I divided the API into separate route groups for authentication, emissions, uploads, anomalies, reports, facilities, activity records, calculations, recommendations, and scenarios. This keeps each area of functionality easier to follow than it would be in one large controller file. I also added a small `/health` endpoint so Docker and deployment checks can confirm that the service is running.

5.2.2 Configuration Management

Runtime configuration is handled in `backend/config.py` with Pydantic settings. The backend reads values from environment variables and, during local development, from an optional `.env` file. The main settings include the database URL, CORS origins, JWT secret, JWT algorithm, access-token expiry, refresh-token expiry, and application environment.

I added a production safety check to avoid a common deployment mistake. If the backend starts with `ENVIRONMENT=production` while still using the default development JWT secret, startup fails instead of allowing the application to run with insecure credentials.

5.2.3 Database Session and Initialization

The database setup is in `backend/database.py`. SQLAlchemy creates the engine from `DATABASE_URL`, defines the shared declarative base, and exposes `SessionLocal` for request-scoped database sessions. The `get_db()` dependency opens a session for an API request and closes it after the request has finished.

During startup, `init_db()` imports the model package, creates the declared tables, and seeds the default roles: `admin`, `facility_manager`, and `auditor`. This gives a new deployment the basic role structure it needs before users start working with the system.

5.2.4 Data Model Implementation

The backend models cover the main records needed for a carbon-monitoring platform. I used SQLAlchemy typed mappings so the database schema is clear in the Python code and stays close to the domain objects used by the API.

Table 5.2: Implemented backend data models

Model	Purpose
`Facility`	Stores monitored site details, including name, location, region code, coordinates, and facility type.
`User`, `Role`, `UserSession`	Store user accounts, access roles, and refresh-token session records used for authentication and revocation.
`Upload`	Tracks uploaded CSV files, the user who uploaded them, facility links, processing status, row count, and upload time.
`ActivityRecord`	Stores measured activity input before emission calculation. Supported activity types include electricity, diesel, petrol, and LPG, with sources such as manual entry, CSV upload, and OCR review.
`EmissionFactor`	Stores lookup multipliers by activity type, region, unit, and validity period so calculations can be reproduced later.
`CalculatedEmission`	Stores calculated emission results, including scope, carbon dioxide equivalent in kilograms, factor ID, and calculation time.
`Recommendation`	Stores rule-engine outputs for a facility and reporting period, including the rule ID, recommendation message, and supporting values.
`Scenario`	Stores what-if simulations separately from official reporting records.

5.2.5 Authentication and Authorization

Authentication is spread across `backend/routers/auth.py`, `backend/security.py`, `backend/dependencies.py`, and `backend/services/sessions.py`. I did not include public sign-up because account creation should be controlled by an administrator. The first administrator can be created with `backend/create_admin.py`, which hashes the password and assigns the selected role.

The login flow checks the submitted email and password with bcrypt through Passlib. After a successful login, the backend returns a short-lived JWT access token and sets a secure HTTP-only refresh-token cookie. Refresh tokens are stored only as hashes in the `sessions` table. When `/api/auth/refresh` is called, the previous refresh token is revoked and replaced with a new one, which limits the risk if an old token is exposed. Protected routes use `get_current_user()` to decode and validate the access token. Role-sensitive routes use `require_role()` to limit access to approved roles such as `admin` and `facility_manager`. For example, CSV upload routes are restricted to users who are allowed to manage facility data.

5.2.6 CSV Upload and Data Ingestion

CSV ingestion is handled in `backend/routers/upload.py` and `backend/services/uploads.py`. The upload endpoint receives a file from an authenticated user, reads the file bytes, validates the required columns, creates an `Upload` record, and updates the upload status after processing.

The parser expects `timestamp`, `metric`, and `value` columns. Optional fields such as unit and facility name are kept when they are present. Timestamps are normalized with ISO date parsing, numeric readings are converted to floating-point values, and invalid rows return clear validation errors. Parsed rows also keep their upload source

so they can be compared with manual or OCR-reviewed activity records.
I kept upload validation separate from later activity-record creation and emission calculation. The upload stage is responsible for checking and recording the file metadata, while the reporting services handle the conversion of accepted activity data into calculated emission records.

5.2.7 OCR Draft Extraction

OCR support is represented by `backend/services/ocr.py`. The service defines the expected fields for extracting information from utility bills and similar documents: facility ID, period start, period end, activity type, quantity, and unit. OCR output is treated as a draft for human review and is not written directly into `activity_records`. Only confirmed values should become official activity records, which helps prevent extraction mistakes from entering the reporting data.

5.2.8 Emission Calculation Layer

The calculation logic is kept in `backend/services/calculation.py`. I designed this service as a small calculation boundary: it receives an activity record and an emission factor, then returns a calculated emission without performing database access. This makes the same logic usable for both saved records and scenario simulations.
The calculation flow is:

1. An activity record stores a measured input, such as electricity consumption or diesel usage.
2. A matching emission factor is selected using activity type, region, unit, and validity period.
3. The activity quantity is multiplied by the factor value to compute `co2e_kg`.
4. The emission scope is chosen from the activity type: electricity maps to Scope 2, while direct fuel combustion maps to Scope 1.
5. The result is stored in `calculated_emissions` with references to the source activity record and the factor used.

This design keeps the calculation traceable because each result can be linked back to both the original activity data and the emission factor used for the calculation.

5.2.9 Emission Queries, Anomaly Detection, and Recommendations

Dashboard data is served through `backend/routers/emissions.py`, which exposes latest readings, time-series data, summary metrics, and cross-verification endpoints. The query logic in `backend/services/emissions.py` reads from `activity_records` and `calculated_emissions` to provide PostgreSQL-backed dashboard aggregations.

Anomaly detection is implemented in `backend/services/anomaly.py`. The `detect()` function groups readings by metric and facility, trains an Isolation Forest for each group when there are enough samples, and returns records flagged as anomalies with an anomaly score and expected value. Keeping this logic in a service module makes it easier to test independently from the API routes.
Recommendations are handled in `backend/services/recommendations.py`. The rule engine checks facility and period data for patterns such as Scope 2 dominance, month-over-month spikes, and overlap with anomaly results. The recommendation model and router provide the storage and API boundary for these outputs.

5.2.10 Scenario Simulation

I kept scenario simulation separate from official reporting data. The `Scenario` table stores modified inputs and projected carbon dioxide equivalent values. The service contract in `backend/services/scenario.py` merges scenario changes over a baseline activity record, calls the shared calculation logic, and avoids writing hypothetical values into the official activity or calculated-emission tables. This keeps audit records separate from experimental what-if analysis.

5.2.11 PDF ESG Report Output

The report service in `backend/services/report.py` creates a downloadable PDF with ReportLab. The `/api/reports/esg` endpoint returns the file as `carbontrace-esg-report.pdf`. The report contains a CarbonTrace title, report timestamp, emission summary table, and anomaly table. If there is no data available, the endpoint still returns a valid PDF with an empty-state message.

5.3 Frontend Implementation

5.3.1 Application Routes and Protected Layout

The frontend is a Vite React application. The main route tree is defined in `frontend/src/App.jsx`. The login page is public, while the dashboard pages are wrapped by `ProtectedRoute` and shown inside the shared `Layout`. The protected pages are:

- Live Monitor dashboard at `/`
- Trends and analytics at `/trends`
- Anomaly log at `/anomalies`
- Cross-verification view at `/cross-verify`
- ESG report download page at `/esg-report`

The layout uses reusable sidebar and top-bar components. The sidebar gives access to monitoring, trends, anomalies, cross-verification, and ESG reporting. The top bar shows the CarbonTrace brand, a notification button, the user's initial, and the logout action.

5.3.2 Frontend Authentication Flow

Frontend authentication is managed through `frontend/src/context/AuthContext.jsx` and `frontend/src/api/client.js`. When the application loads, the auth provider tries to refresh the session through `/api/auth/refresh`. If the refresh succeeds, the new access token is kept in memory and the current user is loaded from `/api/auth/me`. If the refresh fails, the user stays unauthenticated and protected routes redirect to the login page.
The Axios client sends cookies with requests and adds the in-memory bearer token to the `Authorization` header. If a request receives a 401 response, the client makes one refresh attempt and then retries the original request. This keeps the user signed in without storing access tokens in local storage.

5.3.3 Dashboard Data Fetching

Dashboard data is fetched with TanStack Query hooks in `frontend/src/api/hooks.js`. These hooks call the backend for summary data, latest emissions, time-series trends, anomalies, and cross-verification results. Summary, latest, and time-series queries refresh every 30 seconds, which gives the dashboard a live-monitoring feel without adding WebSocket complexity.

5.3.4 Page-Level Features

The main frontend pages match the backend API features:

- **Dashboard:** shows the live monitoring overview using summary and latest-emission hooks.
- **Trends:** visualizes time-series values with Recharts and lets the user choose the metric and interval.
- **Anomalies:** lists anomaly records returned by the anomaly API.
- **Cross-Verification:** compares values from different sources to support audit checks.
- **ESG Report:** calls the report endpoint and downloads the PDF report.
- **Login:** provides the CarbonTrace sign-in screen and submits credentials to the auth context.

5.4 Deployment Implementation

5.4.1 Docker Compose Services

The deployment setup is defined in `docker-compose.yml`. The PostgreSQL service stores the relational data and includes a health check with `pg_isready`. The backend service builds from `backend/Dockerfile`, waits for PostgreSQL to become healthy, and exposes port 8000 for local development. The frontend service builds the React application and serves the static files. Nginx acts as the public entry point.

5.4.2 Backend and Frontend Images

The backend image uses the official `uv` Python 3.11 base image. It copies the dependency files first to improve Docker build caching, installs production dependencies with `uv sync`, copies the backend source code, places the project virtual environment on the path, and starts Uvicorn with two workers.
The frontend image uses a multi-stage build. The first stage uses Node 20 to install dependencies and run `npm run build`. The second stage uses Nginx to serve the compiled frontend files from the build output.

5.4.3 Reverse Proxy and Environment Variables

Nginx forwards `/api/` requests to the backend service and sends all other requests to the frontend service. This gives the browser a single origin, which simplifies refresh-token cookies and CORS behavior.
The `.env.example` file lists the runtime configuration needed by the system, including PostgreSQL credentials, JWT settings, CORS origins, API base URL, and HTTP port. The React app reads `VITE_API_BASE_URL`, while the backend reads values such as `DATABASE_URL`, `JWT_SECRET_KEY`, `ACCESS_TOKEN_EXPIRE_MINUTES`, and `REFRESH_TOKEN_EXPIRE_DAYS`.

5.5 Implementation Summary

Overall, CarbonTrace now has the main pieces needed for a secure carbon-emission monitoring and ESG reporting platform. The implementation includes authentication, role seeding, database models, upload validation, frontend routing, a protected dashboard layout, API refresh handling, PDF reporting, Docker deployment, and CI checks. Together, these parts provide a maintainable base for recording activity data, calculating emissions, reviewing anomalies, and producing reports.

Chapter 6

System Testing

System testing verifies that the CarbonTrace application satisfies the functional and non-functional requirements defined for the project. The testing focus is on authentication, database initialization, upload validation, API routing, frontend navigation, report generation, and containerized deployment.

6.1 Testing Strategy

The application is tested at three levels. Backend checks verify that the FastAPI application imports successfully, initializes configuration, creates database models, and exposes expected route groups. Functional checks verify login, token refresh, protected-route behavior, CSV validation, and PDF report generation. Frontend checks verify that the React application builds successfully and that protected pages are reachable through the configured route tree.

6.2 Test Cases

6.3 Development Workflow

Table 6.1: System Test Cases

ID	Area	Test Description	Expected Result
TC1	Authentication	Submit valid administrator credentials through the login API.	Access token is returned and refresh cookie is set.
TC2	Authorization	Access a protected endpoint without a bearer token.	Request is rejected with an authentication error.
TC3	CSV Upload	Upload a CSV file containing the required `timestamp`, `metric`, and `value` columns.	File is parsed and upload metadata is stored.
TC4	CSV Validation	Upload a CSV file with missing required columns or invalid numeric values.	Validation error is returned with a clear reason.
TC5	Dashboard API	Request summary, latest, trend, anomaly, and cross-verification data from the frontend hooks.	API responses match the expected JSON shape.
TC6	Report Download	Call the ESG report endpoint from the frontend report page.	A valid PDF file is returned for download.
TC7	Frontend Routing	Navigate to live monitor, trends, anomalies, cross-verification, and ESG report pages after login.	Protected pages render inside the shared layout.
TC8	Deployment	Start the Docker Compose stack with PostgreSQL, backend, frontend, and Nginx.	Services start in dependency order and route traffic correctly.

Table 6.2: Implementation Workflow

Sl. No.	Work	Duration (Weeks)
1	Requirements analysis and system design	1
2	Backend data models, configuration, and authentication	3
3	Upload validation, calculation boundaries, and report generation	2
4	React dashboard layout, routing, and API integration	3
5	Docker Compose deployment and reverse-proxy configuration	1
6	Testing, documentation, and final report preparation	2

Chapter 7

Results and Discussion

7.1 Application-Level Results

The implemented CarbonTrace application establishes the main workflow required for carbon emission monitoring and ESG reporting. The backend exposes organized API route groups for authentication, facilities, uploads, emissions, activity records, calculations, anomalies, recommendations, scenarios, and reports. The frontend consumes these APIs through a protected React dashboard and presents the main monitoring functions through separate pages.

The completed structure demonstrates that the system can be deployed as an integrated web application rather than as disconnected scripts. PostgreSQL stores the core application data, FastAPI provides the business and security layer, React renders the user interface, and Nginx provides a single browser-facing entry point for frontend and API traffic.

7.2 Backend Results

The backend implementation provides user authentication, role seeding, JWT access tokens, refresh-token rotation, secure password hashing, database session handling, and application startup validation. Upload processing validates CSV files, records upload metadata, and normalizes row data for later activity-record processing. The data model includes the major entities needed for facility-level carbon accounting, including activity records, emission factors, calculated emissions, recommendations, and scenarios.

The report-generation module produces a downloadable ESG PDF with a title, generation timestamp, emission summary table, and anomaly table. This confirms that the reporting surface is in place and can be connected to calculated emission data as the aggregation queries mature.

7.3 Frontend Results

The frontend provides a protected dashboard layout with navigation for live monitoring, trend analysis, anomaly review, cross-verification, and ESG reporting. Authentication state is managed in memory, while refresh tokens are handled through secure cookies. The Axios client retries a failed request once after refreshing the access token, which keeps normal user sessions active without storing access tokens in browser local storage. TanStack Query is used for dashboard data fetching and periodic refresh. Recharts supports visual trend presentation, while Tailwind CSS keeps the interface consistent across the main screens.

7.4 Deployment Results

The Docker Compose configuration runs PostgreSQL, backend, frontend, and Nginx as coordinated services. The backend waits for the database health check before startup, and Nginx routes API traffic and frontend requests through one origin. This deployment structure simplifies browser security configuration, especially for refresh-token cookies and cross-origin API calls.

7.5 Discussion

The current implementation provides a strong base for an auditable carbon monitoring platform. The most important completed outcomes are the secure application structure, normalized database model, upload boundary, protected dashboard, report-generation endpoint, and containerized deployment. The next development step is to complete the end-to-end activity pipeline by creating activity records from all accepted sources, selecting matching emission factors, persisting calculated emissions, and exposing PostgreSQL-backed dashboard aggregations.

Chapter 8

Conclusion and Future Work

CarbonTrace demonstrates a practical full-stack approach for automated carbon emission monitoring and ESG reporting. The project replaces fragmented manual reporting workflows with a centralized application that manages users, facilities, upload metadata, activity records, emission factors, calculated emissions, anomalies, recommendations, scenarios, and reports. The FastAPI backend, PostgreSQL database, React dashboard, and Docker-based deployment provide a maintainable foundation for secure and auditable environmental data management.

The implementation also shows the value of separating major responsibilities across clear layers. Authentication and role-based access are handled by the backend security modules, data persistence is handled through SQLAlchemy models and PostgreSQL, dashboard interactions are handled by the React frontend, and report generation is handled by a dedicated PDF service. This structure makes the system easier to extend without disrupting existing components.

Future work can further strengthen the platform in the following ways:

- Complete the end-to-end activity-record pipeline for manual entries, CSV uploads, and reviewed OCR outputs.
- Add PostgreSQL aggregation queries for dashboard summaries, trends, cross-verification, and anomaly history.
- Persist anomaly-detection results and connect them to recommendation rules for facility-level action plans.
- Expand the ESG report templates to cover organization-specific and regulator-specific disclosure formats.
- Add direct IoT or meter integrations once the core application workflow is stable.

Overall, the project provides a clear and extensible foundation for transparent carbon accounting. With the remaining calculation and analytics pipeline completed, CarbonTrace can support reliable ESG reporting and help organizations make better decisions about emission reduction.

References

[1] J. O. Ojadi, E. C. Onukwulu, C. S. Odionu, and O. A. Owulade, "Leveraging IoT and deep learning for real-time carbon footprint monitoring and optimization in smart cities and industrial zones," *Iconic Research and Engineering Journals*, vol. 6, no. 11, pp. 946–964, 2023. [Online]. Available: https://www.irejournals.com/paper-details/1704555

[2] C. D. Fay, B. Corcoran, and D. Diamond, "Green IoT event detection for carbon-emission monitoring in sensor networks," *Sensors*, vol. 24, no. 1, p. 162, 2024. DOI: 10.3390/s24010162

[3] C. Badii, S. Bilotta, D. Cenni, A. Difino, P. Nesi, I. Paoli, and M. Paolucci, "Estimating CO_2 emissions from IoT traffic flow sensors and reconstruction," *Sensors*, vol. 22, no. 9, p. 3382, 2022. DOI: 10.3390/s22093382

[4] R. Suryawanshi et al., "IoT based real-time environment monitoring and safety for factory workplace," in *Proceedings of the International Conference on Communication, Computing and Ubiquitous Technology (IC-CGU 2024)*, IEEE, 2024. DOI: 10.1109/ic-cgu58078.2024.10530721

[5] M. Salehi, *The power of IoT in ESG data collection and analysis*, SSRN Working Paper, Jul. 2023. DOI: 10.2139/ssrn.4501798 [Online]. Available: https://ssrn.com/abstract=4501798

[6] M. Bajer, "Building an IoT data hub with Elasticsearch, Logstash and Kibana," in *2017 5th International Conference on Future Internet of Things and Cloud Workshops (FiCloudW)*, IEEE, 2017, pp. 63–68. DOI: 10.1109/FiCloudW.2017.113

[7] T. T. T. Ngo, D. Sarramia, M.-A. Kang, and F. Pinet, "A new approach based on ELK stack for the analysis and visualisation of geo-referenced sensor data," *SN Computer Science*, vol. 4, no. 3, p. 241, 2023. DOI: 10.1007/s42979-022-01628-6

[8] A. Narute, S. Navagire, and S. Konge, "Real-time air quality monitoring using IoT sensors," *International Journal for Research Trends and Innovation*, vol. 10, no. 4, 2025, ISSN: 2456-3315. [Online]. Available: https://www.ijrti.org/papers/IJRTI2504133.pdf

[9] S. Winter et al., *Using the integral digital twin for product carbon footprint calculation*, ScienceDirect, 2025. [Online]. Available: https://doi.org/10.1016/j.susmat.2025.e01234

[10] C. Li and W. Lao, "Internet of things' sustainability effects: Quantile and temporal insights," *Humanities and Social Sciences Communications*, vol. 12, no. 1, p. 396, 2025. DOI: 10.1057/s41599-025-04665-7

Appendix A

Screenshots

This appendix is reserved for application screenshots captured from the deployed CarbonTrace dashboard. The main screens to include are the login page, live monitoring dashboard, trends page, anomaly log, cross-verification page, and ESG report download page.

Appendix B

User Manual

B.1 Starting the Application

The application is designed to run through Docker Compose. After configuring the required environment variables, start the stack so that PostgreSQL, the backend, the frontend, and Nginx run together. Nginx serves the frontend and forwards `/api/` requests to the backend.

B.2 Signing In

Open the CarbonTrace web application in a browser and sign in using an administrator or assigned user account. Public registration is not enabled. An administrator account must be created through the backend administration script before normal use.

B.3 Using the Dashboard

After login, use the sidebar to move between the main pages:

- **Live Monitor:** view current emission summaries and latest readings.
- **Trends:** inspect historical emission patterns by metric and interval.
- **Anomalies:** review records flagged as unusual by the anomaly service.
- **Cross-Verification:** compare values from different data sources for audit checks.
- **ESG Report:** download the generated PDF report.

B.4 Uploading Data

Authorized users can upload CSV files through the backend upload workflow. Each file should include the required columns `timestamp`, `metric`, and `value`. Optional fields such as unit and facility name may be included when available. Invalid files are rejected with validation messages so that the user can correct the source data.

B.5 Signing Out

Use the logout option in the top bar to end the session. The frontend clears the in-memory access token, and the backend session can be refreshed or revoked through the authentication flow.